

SOURCE CODE DOCUMENTATION

Project Performance Improvement Toolkit

Maintenance guide for the main app

Technical source: oli_v6_deploy.py

Updated: May 11, 2026

1. Source Code Scope

The documented code is oli_v6_deploy.py. The project also contains oli_v6_deploy_core.py and oli_v6_deploy_recommendations.py, but this documentation does not infer behavior from them.

Maintenance rule: when the main app changes, review whether the same change must be propagated to the separate variants before delivering an operational version to the ILO team.

2. Relevant Directory Structure

File	Operational Role
oli_v6_deploy.py	Main Streamlit application and source of truth for this documentation.
requirements.txt	Python dependencies used by the app.
df_complete_all_full.xlsx	Main enriched recommendation dataset.
df_split_actions.xlsx	Raw/disaggregated dataset used for recommendations and years.
analyzed_recommendations_plans_v5.csv	Additional analysis of recommendations and action plans; pipe-separated.
emb_Recomm_rec_cl_4.pt	Recommendation embeddings loaded at startup for the FAISS index.
Recommendation_RAG_Metadata.pt	Metadata aligned to the recommendation embeddings loaded at startup.
Appraisal Checklist_2025 es-419.xlsx	Preliminary appraisal questions, rubric sheet.
Rubricas_6ago2025.xlsx	Specific-attribute rubrics used by Tab 2.
Evaluacion de sostenibilidad del proyecto_rubric_9feb26.xlsx	Active sustainability rubric for Tab 3.
Recommendations_World.xlsx	World recommendations for classification in Tabs 5 and 6.
Frame_Recommendations_English.xlsx	Dimension/subdimension classification frame.

3. Local Installation

1. Use a Python version compatible with the project's pinned dependencies.
2. Create and activate a virtual environment.
3. Install dependencies with `pip install -r requirements.txt`.
4. Configure OPENAI_API_KEY as an environment variable.

5. Run `streamlit run oli_v6_deploy.py` from the project directory.

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run oli_v6_deploy.py
```

OPENAI_API_KEY must be configured in the environment or secret manager selected by ILO. Its real value must not be committed or shared in documentation.

4. Main Function Map

Block	Functions or Classes	Description
Excel/export	to_excel	Converts DataFrames to XLSX with <code>xlsxwriter</code> and disables <code>strings_to_urls</code> .
Tokens	truncate_to_token_limit	Cuts text to token limits using <code>tiktoken</code> .
Embeddings and RAG	get_embedding_with_retry, find_similar_recommendations, find_recommendations_by_term_matching, load_embeddings	Generates embeddings and loads a FAISS index at startup; the visible recommendation-search UI is commented out, but the dependency is still active.
Document chat	Tab 4, truncate_to_token_limit, client.embeddings.create, faiss.IndexFlatIP	Uses full context for small texts and RAG with session-cached chunks for large texts.
Recommendation analysis	AnalysisCache, analyze_recommendation_plan_pair, run_row_analysis, generate_executive_summary	Evaluates plan coherence, quality, feasibility, impact, innovation, and generates an executive summary; the effective deep-analysis model is <code>gpt-4o-mini</code> even though the UI shows a selector.
Data loading	prepare_additional_data, load_data, load_extended_data	Normalizes columns, years, categories, and merges additional analysis.
Document extraction	extract_docx_structure_enhanced, validate_extraction, extract_document_content	Extracts hierarchy, tables, metrics, and text from DOCX.
Preliminary appraisal	load_appraisal_questions, analyze_question_with_llm_tab1, detect_transversal_matter, _critic_impl, _apply_critic_gate_and_render, _apply_transversal_gate_and_render, synthesize_subsection_analysis, synthesize_section_analysis, create_results_download_with_sections	Evaluates question by question, applies the A-E critique or transversal A/B rule, and creates multilevel XLSX/ZIP outputs.
Sustainability and attributes	evaluate_criterion_with_llm, synthesize_evaluations	Evaluates criteria with score, analysis, and evidence.
Classification	EmbeddingsCache, classify_recommendations,	Classifies recommendations against the frame by <code>numpy</code>

	classify_recommendations_en, verify_match_with_llm	cosine similarity, with persistent cache and optional LLM verification.
--	---	---

5. State Conventions

- Each tab uses its own `st.session_state` keys, such as `tab1_results_df`, `document_extracted_tab2`, `tab3_results`, `doc_chat_docs`, `doc_chat_embeddings`, `classified_world_df`, and `deep_analysis_df`.
- Results persist during the session and can be cleared with `clear-results` buttons.
- New file uploads are detected with `hash(uploaded_file.getvalue())` and reset extraction state when needed.
- Exception: Tab 4 detects document changes by file-name list, not by content hash.
- Downloadable outputs are generated in memory with `BytesIO`; they are not automatically saved to disk.

5.1 Special Logic for Two-Part Questions

`parse_two_part_question` separates compound questions and privileges Part 2.

`analyze_question_with_llm_tab1` instructs the model to start reasoning with 'Se identificaron 2 partes en esta pregunta.' and to use Part 1 only as context. `_critic_impl` repeats the evaluation with the full document and `_apply_critic_gate_and_render` adjusts the verdict based on the count of dedicated elements.

The system distinguishes [DEDICATED] evidence from [FRAMING] evidence. Framing evidence, such as broad group lists or general inclusion language, cannot justify Partial or Yes by itself.

5.2 Special Logic for Transversal Matters

TRANSVERSAL_MATTERS is an in-code dictionary that recognizes gender, non-discrimination, disability, social dialogue and tripartism, and environmental sustainability through accent-stripped normalized aliases. When `detect_transversal_matter` finds one of these matters in a question, `_critic_impl` uses `CRITIC_SCHEMA_TRANSVERSAL` and `_apply_transversal_gate_and_render` instead of the general A-E schema.

The reasoning output for these cases begins with an audit line for transversal criteria: A means objective/output/activity and B means budget. The applied rule is `Yes=A+B`, `Partial=A or B`, `No=neither A nor B`. Indicators and targets do not count for the transversal rule.

6. Error Handling

- Missing critical files: the app shows `st.error` or `st.warning` and stops that tab's flow when the resource is mandatory.
- Missing `OPENAI_API_KEY`: the app shows a global warning and OpenAI-dependent functions return an error or warning.
- AI analysis errors: errors are captured and returned in Error columns or chat messages.
- Invalid JSON responses: several flows attempt to strip Markdown fences; if parsing still fails, the app returns score 0 or a default score with an explanatory error.
- Rate limits: some evaluations retry with backoff; in other cases concurrency is reduced by design.

7. Recommended Tests and Verification

- Run `test_two_part_parsing.py` when changing `parse_two_part_question` or two-part question logic.

- Manually test one question for each transversal matter to confirm alias detection and A/B rule application.
- Manually test a DOCX with Heading 1/2 and tables in Tabs 1, 2, and 3.
- Test chat with a small TXT and a large DOCX to validate the two paths: full context and RAG; confirm that changing content while keeping the same file name requires restarting the session or renaming the file.
- Test Tab 5 on a limited sample with LLM Verification off and on.
- Verify each XLSX/ZIP download opens correctly and preserves evidence columns.

8. Visible Technical Debt

- Large commented blocks from older versions are non-functional but make maintenance harder.
- Data and embedding initialization appears twice in the main block.
- Tab 4 promises traceability by section/page, but DOCX extraction for chat only reads paragraphs with python-docx; it does not guarantee tables, pages, or hierarchical metadata.
- Tabs 5 and 6 have a model selector for deep analysis that does not control the effective model; run_row_analysis uses gpt-4o-mini.
- Some Tab 3 error messages still mention PRODOC_rubric.xlsx even though the active rubric is Evaluacion de sostenibilidad del proyecto_rubric_9feb26.xlsx.
- There is a commented legacy Tab 2 block that also loads gender_rubric; it is not the active flow. The active implementation loads rubric_gender_ and exposes it as gender integration.
- TRANSVERSAL_MATTERS aliases are defined in code; if ILO changes official terminology or adds matters, that dictionary and Tab 1 tests must be updated.
- lessons_metadata.pt is expected by load_lessons_embeddings, but it was not present in the observed local inventory; if that functionality is activated, the file must be supplied.
- The app does not clearly separate interface, domain, and external-service layers; large changes require manual tests by tab.